Kohenen Neural Network

For Unsupervised Learning Data

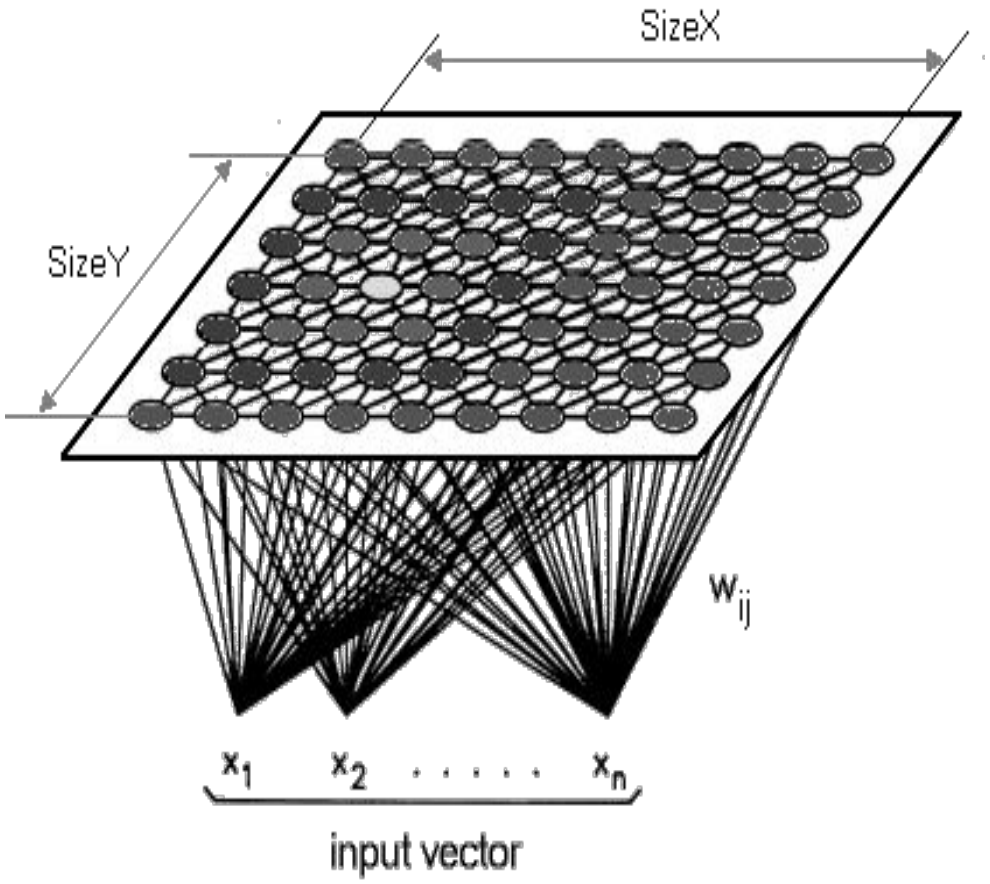
Introduction

1. Kohenen, a self-organising map is an unsupervised learning model, intended for applications in which *maintaining* a topology between input and output spaces is of importance.
2. The notable characteristic of this algorithm is that the input vectors that are close — similar — in high dimensional space are also mapped to nearby nodes in the 2D space.
3. It is in essence a method for dimensionality reduction, as it maps high-dimension inputs to a low (typically two) dimensional discretised representation and conserves the underlying structure of its input space.

1. A valuable detail is that the entire learning occurs without supervision i.e. the nodes are *self-organising*.
2. They are also called *feature maps*, as they are essentially retraining the features of the input data, and simply grouping themselves according to the *similarity* between one another.
3. This has a pragmatic value for visualising complex or large quantities of high dimensional data and representing the relationship between them into a low, typically two-dimensional, field to see if the given unlabelled data has any structure to it.

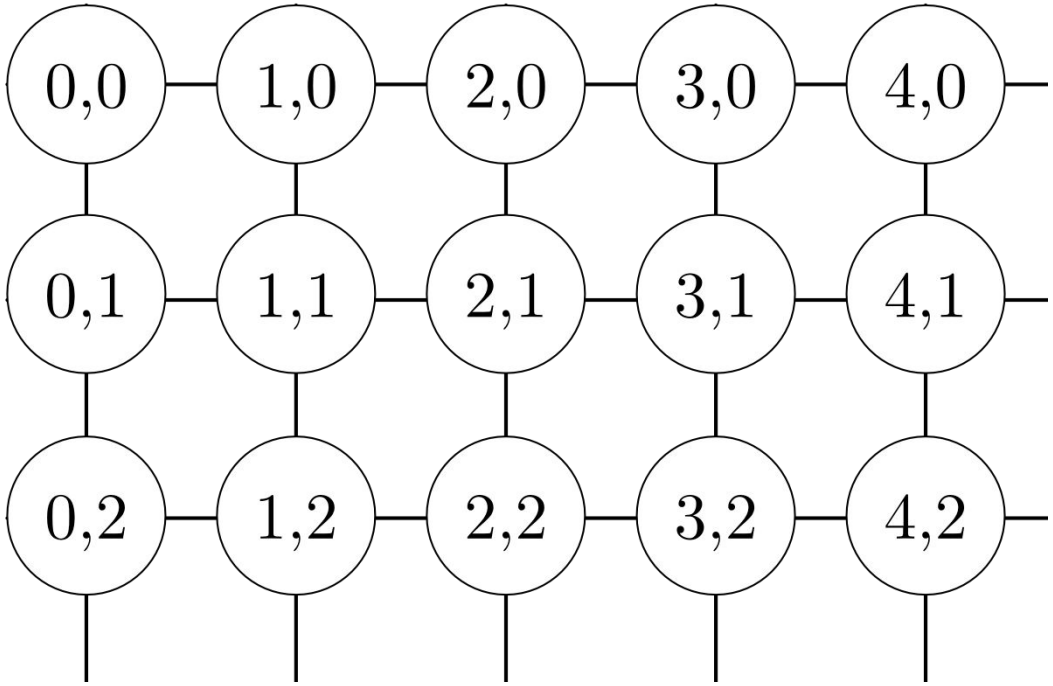
Structure

1. A Self-Organizing Map (SOM) differs from typical ANNs both in its architecture and algorithmic properties. Firstly, its structure comprises of a single-layer linear 2D grid of neurons, instead of a series of layers.
2. All the nodes on this grid are connected directly to the input vector, *but not to one another*, meaning the nodes do not know the values of their neighbours, and only update the weight of their connections as a function of the given inputs.
3. The grid *itself is the map* that organises itself at each iteration as a function of the input of the input data. As such, after clustering, each node has its own *(i,j)* coordinate, which allows one to calculate the Euclidean distance between 2 nodes by means of the Pythagorean theorem.

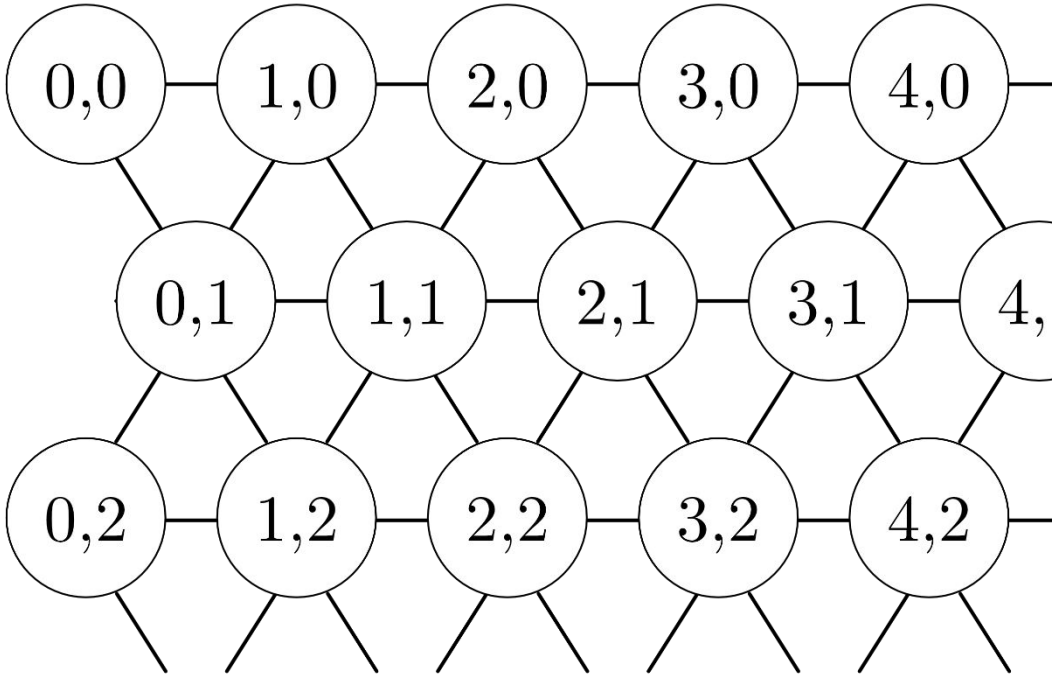


Structure

Rectangular Structure



Hexagonal Structure



Properties

1. A Self-Organising Map, additionally, uses competitive learning as opposed to error-correction learning, to adjust it weights.
2. *Only a single node* is activated at each iteration in which the features of an instance of the input vector are presented to the neural network, as all nodes compete for the right to respond to the input.
3. The chosen node — the Best Matching Unit (BMU) — is selected according to the similarity, between the current input values and all the nodes in the grid.
4. The node with the smallest Euclidean difference between the input vector and all nodes is chosen, *along with its neighbouring nodes* within a certain radius, to have their position slightly adjusted to match the input vector.
5. By going through all the nodes present on the grid, the entire grid eventually matches the complete input dataset, with similar nodes grouped together towards one area, and dissimilar ones separated.

Variables List

- *t* is the current iteration
- *n* is the iteration limit, i.e. the total number of iterations the network can undergo
- λ is the time constant, used to decay the radius and learning rate
- *i* is the row coordinate of the nodes grid
- *j* is the column coordinate of the nodes grid
- *d* is the distance between a node and the BMU
- *w* is the weight vector
- *w_ij(t)* is the weight of the connection between the nodes *i,j* in the grid, and the input vector's instance at the iteration *t*
- *x* is the input vector
- *x(t)* is the input vector's instance at iteration *t*
- *α(t)* is the learning rate, decreasing with time in the interval [0,1], to ensure the network converges.
- *β_ij(t)* is the neighbourhood function, monotonically decreasing and representing a node *i, j*'s distance from the BMU, and the influence it has on the learning at step *t*.
- *σ(t)* is the radius of the neighbourhood function, which determines how far neighbour nodes are examined in the 2D grid when updating vectors. It is gradually reduced over time.

Algorithm

1. Initialize each node's weight *w_ij* to a random value
2. Select a random input vector *x_k*
3. Repeat point 4. and 5. for all nodes in the map:
4. Compute Euclidean distance between the input vector *x(t)* and the weight vector *w_ij* associated with the first node, where *t, i, j = 0.*
5. Track the node that produces the smallest distance *t*.
6. Find the overall Best Matching Unit (BMU), i.e. the node with the smallest distance from all calculated ones.
7. Determine topological neighborhood *βij(t)* its radius *σ(t)* of BMU in the Kohonen Map
8. Repeat for all nodes in the BMU neighborhood: Update the weight vector *w_ij* of the first node in the neighborhood of the BMU by adding a fraction of the difference between the input vector *x(t)* and the weight *w(t)* of the neuron.
9. Repeat this whole iteration until reaching the chosen iteration limit *t=n*

Step 1 is the **initialisation** phase, while step 2–9 represent the **training** phase.

Weight Update Formula For Neighbors

$$w_{ij}(t+1) = w_{ij}(t) + \alpha_i(t)[x(t) - w_{ij}(t)], \quad \text{or}$$
$$w_{ij}(t+1) = w_{ij}(t) + \alpha_i(t)\beta_{ij}(t)[x(t) - w_{ij}(t)]$$

The Radius and the Learning rate

$$\sigma(t) = \sigma_0 \cdot \exp(\frac{-t}{\lambda}), \quad \text{where } t = 1, 2, 3 \ldots n$$

$$\alpha(t) = \alpha_0 \cdot \exp(\frac{-t}{\lambda}), \quad \text{where } t = 1, 2, 3 \ldots n$$

The Neighborhood Function Influence

$$\beta_{ij}(t) = \exp\left(\frac{-d^2}{2\sigma^2(t)}\right), \quad \text{where } t = 1, 2, 3 \ldots n$$

BMU (Best Matching Unit) selection

$$d = \min(||\vec{x} - \vec{w_{ij}}||) = \min(\sqrt{\sum_{t=0}^{n}[\vec{x}(t) - \vec{w}_{ij}(t)]^2})$$